

GUIDE D'APPRENTISSAGE ET DE TRAVAIL PRATIQUE

MINI MOTEUR MAPREDUCE AVEC MPI

Implementation de Map, Shuffle, Reduce et comparaison avec Apache Spark

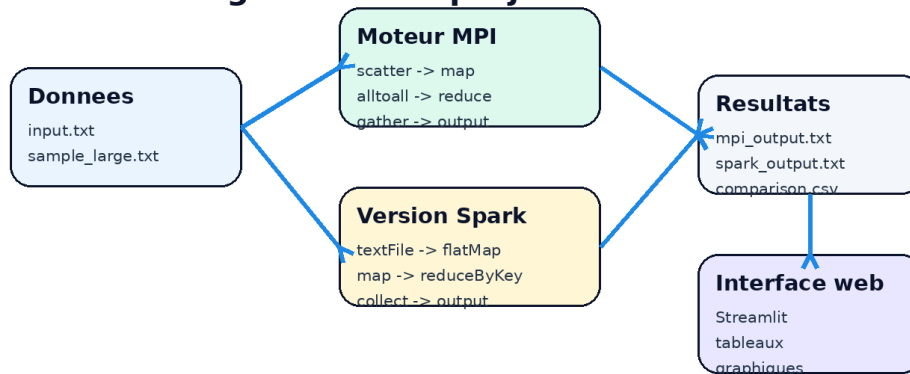
Module : Systemes repartis et programmation parallele

Universite : Universite Hassan II

Annee universitaire : 2025-2026

Element	Information
Realise par	ERFOUDI Anass
	JOUNAIDI Oussama
	MOUSDIK Ismail

Architecture generale du projet



Le meme fichier texte est traite par deux approches, puis les resultats sont compares et visualises.

Vue globale du projet : donnees, traitements MPI/Spark, resultats et interface web.

Sommaire

- 1. Introduction et objectifs du travail pratique
- 2. Concepts theoriques cles
- 3. Architecture du projet
- 4. Preparation de l'environnement
- 5. Implementation pas-a-pas du mini moteur MapReduce
- 6. Version Apache Spark
- 7. Interface web Streamlit
- 8. Protocole experimental et analyses
- 9. Analyse de scalabilite
- 10. Travail demande aux etudiants
- 11. Questions de comprehension
- 12. Conclusion et perspectives
- Annexes : commandes, resultats et captures d'ecran

Objectif du document : Ce document n'est pas seulement un rapport. C'est un TP guide : il explique les notions, donne les commandes, propose les manipulations, puis demande aux étudiants d'analyser les résultats.

1. Introduction et objectifs du travail pratique

Dans les systèmes modernes, les données peuvent devenir trop volumineuses pour être traitées efficacement par un seul programme séquentiel. Le traitement parallèle consiste à diviser le travail entre plusieurs processus afin de réduire le temps d'exécution et d'exploiter plusieurs cœurs ou machines.

Ce TP propose d'implémenter un mini moteur MapReduce avec MPI appliqué au problème classique du Word Count. Les étudiants observent explicitement les phases Map, Shuffle et Reduce, puis comparent cette implémentation manuelle avec une version Apache Spark de plus haut niveau.

Objectifs pédagogiques : Comprendre le principe MapReduce, utiliser mpi4py, manipuler les communications scatter/alltoall/gather, comparer MPI et Spark, et présenter les résultats avec Streamlit.

Compétence visée	Ce que l'étudiant doit savoir faire
Traitement parallèle	Diviser un fichier texte entre plusieurs processus MPI.
MapReduce	Identifier et expliquer Map, Shuffle et Reduce.
MPI	Utiliser scatter, alltoall et gather dans un exemple concret.
Spark	Exécuter un Word Count avec reduceByKey et comparer avec MPI.
Analyse expérimentale	Lire un fichier comparison.csv et interpréter les temps d'exécution.
Démonstration	Utiliser une interface Streamlit pour présenter le projet.

2. Concepts theoriques cles

2.1 Principe de MapReduce

MapReduce est un modele de programmation qui permet de traiter des donnees en deux grandes operations logiques : transformer les donnees sous forme de paires intermediaires, puis regrouper et agreger ces paires. Dans ce TP, la cle est le mot et la valeur est son nombre d occurrences.

Flux MapReduce implemente avec MPI



Objectif pedagogique : rendre visibles les etapes internes que Spark gere automatiquement.

Figure 1 - Flux MapReduce realise par le projet MPI.

2.2 Phase Map

La phase Map est executee localement par chaque processus MPI. Chaque processus recoit une partie du fichier texte, nettoie les mots, puis produit un dictionnaire local des occurrences.

Extrait de code : phase Map

```
def local_map(lines: List[str]) -> Dict[str, int]:
    """Map phase: each MPI process counts words in its own lines."""
    counter: Counter[str] = Counter()
    for line in lines:
        counter.update(tokenize_text(line))
    return dict(counter)
```

Observation attendue : Avant le Shuffle, un meme mot peut exister dans plusieurs dictionnaires locaux. Le resultat n est donc pas encore global.

2.3 Phase Shuffle

La phase Shuffle est l'étape la plus importante du TP. Elle garantit que toutes les occurrences d'un même mot sont envoyées vers le même processus réducteur. Le projet utilise une fonction de hachage stable afin d'obtenir le même résultat dans tous les processus.

Extrait de code : hachage stable et redistribution

```
def stable_word_hash(word: str) -> int:
    """Return the same hash value for a word in every MPI process."""
    digest = hashlib.md5(word.encode("utf-8")).hexdigest()
    return int(digest, 16)

def build_shuffle_buckets(local_counts: Dict[str, int], size: int) -> List[Dict[str, int]]:
    """Shuffle phase: send each word to hash(word) % number_of_processes."""
    buckets: List[Dict[str, int]] = [defaultdict(int) for _ in range(size)]
    for word, count in local_counts.items():
        destination_process = stable_word_hash(word) % size
        buckets[destination_process][word] += count
    return [dict(bucket) for bucket in buckets]
```

Pourquoi utiliser $\text{hash}(\text{mot}) \% \text{nombre_processus}$? : Cette formule transforme un mot en numéro de processus. Ainsi, tous les processus envoient le même mot au même réducteur.

2.4 Phase Reduce

La phase Reduce additionne les valeurs reçues pour chaque mot. À la fin, chaque processus possède une partie du résultat final. Le rang 0 rassemble les dictionnaires réduits avec `gather` et écrit le fichier de sortie.

Extrait de code : phase Reduce

```
def reduce_counts(received_buckets: List[Dict[str, int]]) -> Dict[str, int]:
    """Reduce phase: aggregate all counts received for this reducer process."""
    reduced: Counter[str] = Counter()
    for bucket in received_buckets:
        reduced.update(bucket)
    return dict(reduced)
```

2.5 MPI et communications collectives

Operation MPI	Rôle dans le TP	Moment d'utilisation
scatter	Distribuer les chunks de lignes	Début du traitement
alltoall	Echanger les buckets entre tous les processus	Phase Shuffle
gather	Rassembler les résultats réduits vers le rang 0	Fin du traitement

2.6 Apache Spark

Apache Spark est un framework Big Data de haut niveau. Contrairement à MPI, Spark cache la majorité des détails de communication. Le développeur écrit une chaîne de transformations, et Spark gère la planification, le partitionnement et la réduction.

Extrait de code : pipeline Spark Word Count

```
lines = spark.sparkContext.textFile(input_file)
counts = (
    lines.flatMap(lambda line: line.split())
    .map(clean_spark_word)
    .filter(lambda word: word != "")
    .map(lambda word: (word, 1))
    .reduceByKey(lambda left, right: left + right)
    .collect()
)
```

3. Architecture du projet

Structure du projet

```
mini-mapreduce-mpi-web/
├── data/
│   ├── input.txt
│   └── sample_large.txt
├── src/
│   ├── mini_mapreduce.py
│   ├── spark_wordcount.py
│   ├── benchmark.py
│   └── utils.py
├── web/
│   └── app.py
├── results/
│   ├── mpi_output.txt
│   ├── spark_output.txt
│   └── comparison.csv
├── docs/
│   └── project_explanation.md
├── requirements.txt
├── README.md
└── .gitignore
```

Fichier	Role
src/mini_mapreduce.py	Implementation MPI : lecture, scatter, Map, Shuffle, Reduce, gather et écriture du résultat.
src/spark_wordcount.py	Implementation Apache Spark du même Word Count pour comparaison.
src/benchmark.py	Execution automatique de MPI avec 2, 4 et 6 processus et de Spark local[*].
src/utils.py	Fonctions communes : nettoyage des mots, tokenisation, lecture/écriture, découpage des lignes.
web/app.py	Interface Streamlit : upload, boutons d'exécution, tableaux et graphiques.
results/comparison.csv	Tableau des temps d'exécution obtenu par le benchmark.

4. Preparation de l'environnement

Prerequis : Ubuntu Linux, Python 3, Open MPI, Java JDK 17 pour Spark, et un environnement virtuel Python.

Commandes d'installation

```
sudo apt update
sudo apt install python3 python3-pip python3-venv openmpi-bin libopenmpi-dev openjdk-17-jdk -y

python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Verification : Le prompt du terminal doit afficher (venv). Ensuite, les commandes mpirun, python3 et streamlit doivent être reconnues.

5. Implementation pas-a-pas du mini moteur MapReduce

5.1 Lecture et decoupage du fichier

Le processus de rang 0 lit le fichier d'entree, separe le texte en lignes, puis decoupe ces lignes en chunks equilibres. Chaque processus recoit ensuite un chunk avec scatter.

Extrait : lecture et scatter

```
if rank == 0:
    text = read_text_file(input_file)
    lines = text.splitlines()
    chunks = chunk_lines(lines, size)
else:
    chunks = None

local_lines = comm.scatter(chunks, root=0)
```

Travail a faire : Ouvrir src/mini_mapreduce.py et localiser les lignes qui correspondent a la lecture, au decoupage et au scatter.

5.2 Comptage local : Map

Chaque processus utilise tokenize_text pour nettoyer les mots puis compter ses propres occurrences. Cette etape ne necessite pas de communication.

Travail a faire : Modifier temporairement le code pour afficher le rang du processus et son dictionnaire local sur un petit fichier input.txt.

5.3 Redistribution : Shuffle

Les mots sont places dans des buckets. Le bucket d'un mot est choisi par `stable_word_hash(word) % size`. Ensuite, `alltoall` echange les buckets entre tous les processus.

Extrait : communication alltoall

```
outgoing_buckets = build_shuffle_buckets(local_counts, size)
incoming_buckets = comm.alltoall(outgoing_buckets)
```

Observation attendue : Sans Shuffle, deux processus pourraient compter le meme mot separement. Avec Shuffle, un meme mot arrive chez un seul reducteur.

5.4 Aggregation finale : Reduce et Gather

Chaque reduceur additionne les valeurs recues. Le rang 0 recupere les dictionnaires reduits et produit le fichier final trie par nombre d occurrences.

Extrait : reduce, gather et ecriture

```
reduced_counts = reduce_counts(incoming_buckets)
gathered_counts = comm.gather(reduced_counts, root=0)

if rank == 0:
    final_counts = Counter()
    for partial_counts in gathered_counts:
        final_counts.update(partial_counts)
    write_word_counts(dict(final_counts), output_file)
```

6. Commandes d execution

Commandes principales

```
# Lancer MPI avec 2 processus
mpirun -np 2 python3 src/mini_mapreduce.py data/input.txt results/mpi_output_2.txt

# Lancer MPI avec 4 processus
mpirun -np 4 python3 src/mini_mapreduce.py data/input.txt results/mpi_output_4.txt

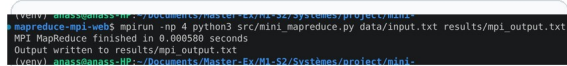
# Lancer MPI avec 6 processus
mpirun -np 6 python3 src/mini_mapreduce.py data/input.txt results/mpi_output_6.txt

# Lancer Spark
python3 src/spark_wordcount.py data/input.txt results/spark_output.txt

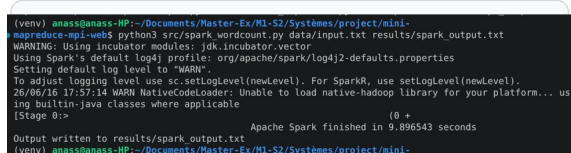
# Lancer le benchmark complet
python3 src/benchmark.py

# Lancer l interface web
streamlit run web/app.py
```

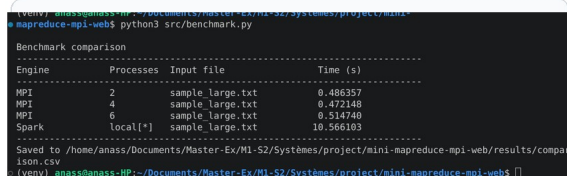
Captures terminal : execution et benchmark



MPI execution in terminal.png



Spark execution in terminal.png



- Benchmark result.png

Figure 2 - Captures terminal : execution MPI, execution Spark et benchmark.

7. Version Apache Spark et interface web

7.1 Version Spark

La version Spark effectue le meme traitement que la version MPI, mais avec des transformations de haut niveau. Le code est plus court car Spark gere automatiquement les partitions et les operations distribuees.

Travail a faire : Comparer le fichier results/mpi_output.txt avec results/spark_output.txt. Les fichiers doivent représenter le meme type de resultat : une ligne par mot et son nombre d occurrences.

7.2 Interface web Streamlit

L interface web permet de presenter le projet sans taper toutes les commandes manuellement. Elle propose un upload de fichier texte, le choix du nombre de processus MPI, les boutons d execution, les tableaux de resultats et les graphiques.

Interface web Streamlit : demonstration

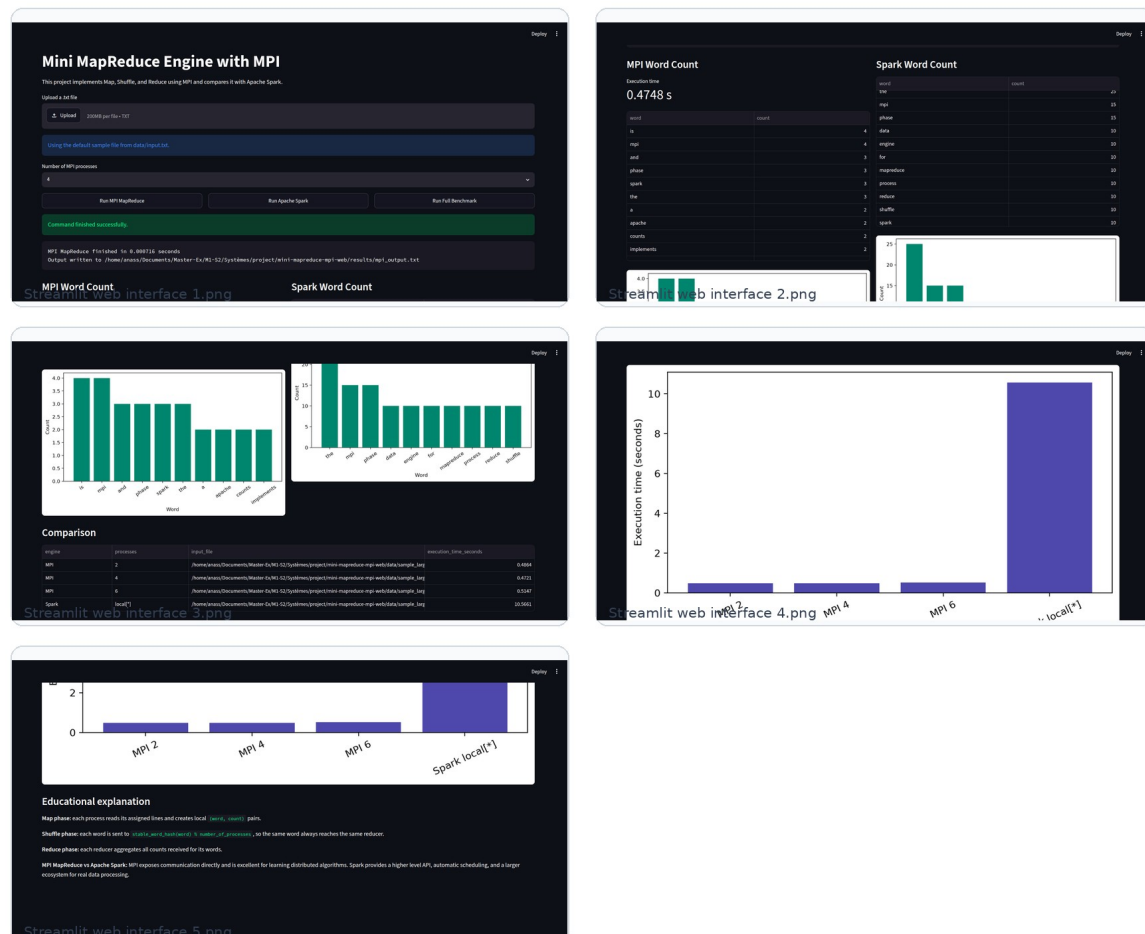


Figure 3 - Interface Streamlit : execution, resultats, graphiques et explication pedagogique.

Element de l interface	Role
Upload .txt	Choisir un fichier texte a traiter.
Number of MPI processes	Tester 2, 4, 6 ou 8 processus MPI.
Run MPI MapReduce	Executer le moteur MPI.
Run Apache Spark	Executer la version Spark.
Run Full Benchmark	Executer toutes les configurations et generer comparaison.csv.
MPI/Spark Word Count	Afficher les mots les plus frequents sous forme de tableau et graphique.

8. Protocole experimental et analyses

Le protocole consiste a utiliser le meme fichier sample_large.txt pour comparer plusieurs configurations. Le script benchmark.py execute MPI avec 2, 4 et 6 processus, puis Spark en mode local[*].

Moteur	Processus	Fichier	Temps	Observation
MPI	2	sample_large.txt	0.486357 s	Reference MPI avec moins de processus.
MPI	4	sample_large.txt	0.472148 s	Meilleur temps observe dans ce test.
MPI	6	sample_large.txt	0.514740 s	Legere hausse due au surcout de communication.
Spark	local[*]	sample_large.txt	10.566103 s	Temps plus eleve sur petit fichier a cause du demarrage Spark/JVM.

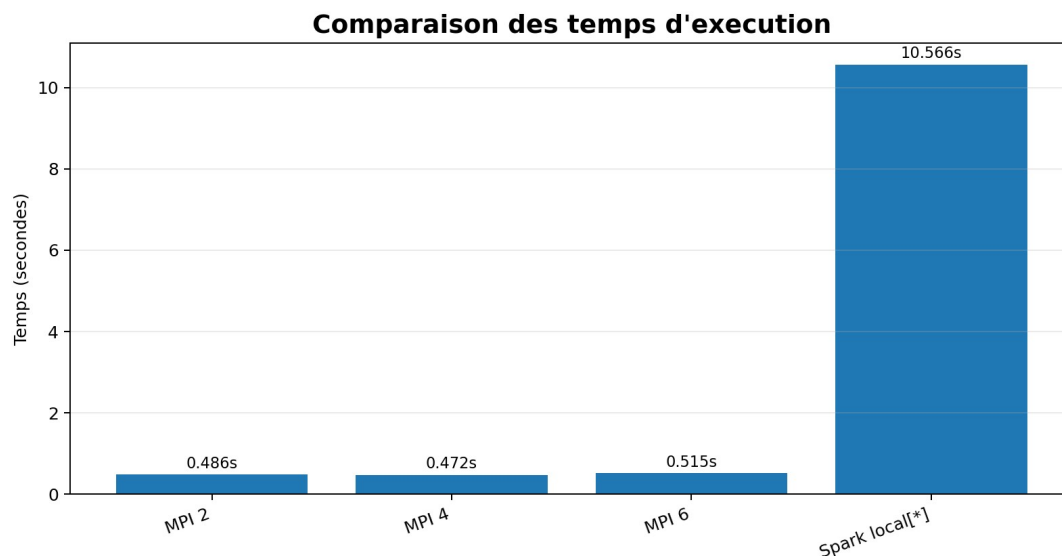


Figure 4 - Graphique du benchmark MPI vs Spark sur sample_large.txt.

Interpretation : Sur ce test, MPI avec 4 processus est legerement plus rapide que MPI avec 2 processus. MPI avec 6 processus devient un peu plus lent, ce qui montre que plus de processus ne garantit pas toujours un meilleur temps. Spark est beaucoup plus lent ici parce que le fichier est petit et que le cout de demarrage du framework domine.

8.1 Resultats Word Count attendus

MPI output - extrait	Spark output - extrait
is 4	the 25
mpi 4	mpi 15
and 3	phase 15
phase 3	data 10
spark 3	engine 10
the 3	for 10
a 2	mapreduce 10
apache 2	process 10
counts 2	reduce 10
implements 2	shuffle 10
map 2	spark 10
mapreduce 2	text 10

9. Analyse de scalabilite

La scalabilite indique comment le temps d execution evolue lorsque le nombre de processus augmente. Dans ce TP, on fait varier le nombre de processus MPI : 2, 4 et 6.

Configuration	Ce qu'il faut observer	Explication possible
MPI 2 processus	Temps de reference.	Peu de communication, mais moins de parallelisme.
MPI 4 processus	Temps souvent meilleur dans ce test.	Bon compromis entre parallelisme et overhead.
MPI 6 processus	Temps legerement plus eleve.	Le surcout de communication peut depasser le gain.
Spark local[*]	Temps plus eleve sur petit exemple.	Demarrage de Spark et de la JVM, framework plus lourd.

Conclusion experimentale : Ce benchmark ne prouve pas que MPI est toujours meilleur que Spark. Il montre surtout que Spark a un cout de demarrage important sur un petit fichier, tandis que MPI est plus leger pour un exemple pedagogique.

10. Travail demande aux etudiants

Etape	Travail a realiser	Livrable attendu
1	Preparer l'environnement Python et installer les dependances.	Capture ou verification des versions.
2	Executer MPI avec 2, 4 et 6 processus.	Fichiers mpi_output_2.txt, mpi_output_4.txt, mpi_output_6.txt.
3	Executer la version Spark.	Fichier spark_output.txt.
4	Executer benchmark.py.	Fichier comparison.csv rempli.
5	Ouvrir Streamlit et lancer les boutons.	Capture de l'interface avec les resultats.
6	Comparer les resultats MPI et Spark.	Tableau de comparaison.
7	Interpreter les temps d execution.	Paragraphe d analyse.
8	Repondre aux questions de comprehension.	Reponses courtes et precises.

Consigne importante : Toutes les commandes doivent etre lancees depuis la racine du projet mini-mapreduce-mpi-web.

10.1 Tableau a remplir par les etudiants

Moteur	Processus	Fichier utilise	Temps observe	Commentaire
MPI	2	sample_large.txt		
MPI	4	sample_large.txt		
MPI	6	sample_large.txt		
Spark	local[*]	sample_large.txt		

10.2 Mini-exercices proposes

- Changer le contenu du fichier data/input.txt et verifier que les resultats changent.
- Ajouter un affichage du rang MPI dans la phase Map pour comprendre la distribution.
- Tester mpirun -np 8 et comparer avec 2, 4 et 6 processus.
- Modifier sample_large.txt en augmentant le nombre de repetitions et relancer le benchmark.
- Ajouter un bouton dans Streamlit pour telecharger le fichier comparison.csv.

11. Questions de comprehension avec reponses attendues

N	Question	Reponse attendue
1	Quel est le role de la phase Map ?	
2	Pourquoi la phase Shuffle est-elle necessaire ?	
3	Pourquoi utiliser une fonction de hachage stable ?	
4	Quel est le role de comm.scatter ?	
5	Quel est le role de comm.alltoall ?	
6	Quel est le role de comm.gather ?	
7	Quelle est la difference principale entre MPI et Spark ?	
8	Pourquoi Spark peut etre lent sur un petit fichier ?	
9	Pourquoi augmenter le nombre de processus ne rend pas toujours le programme plus rapide ?	
10	Quel fichier contient les resultats du benchmark ?	
11	Quel est l'avantage pedagogique de MPI ?	
12	Quel est l'avantage pratique de Spark ?	

12. Conclusion et perspectives

Ce TP permet de comprendre concretement comment un traitement MapReduce peut etre construit a partir de primitives MPI. Les etudiants ne se limitent pas a utiliser un framework existant : ils implementent eux-memes les etapes Map, Shuffle et Reduce, puis comparent leur solution avec Apache Spark.

Les resultats montrent que l'approche MPI est legere et tres utile pour l'apprentissage, tandis que Spark est plus complet et mieux adapte a des environnements Big Data reels. L'interface Streamlit facilite la demonstration et rend le projet plus professionnel.

Perspectives : Ajouter d'autres traitements MapReduce, tester des fichiers plus volumineux, lancer MPI sur plusieurs machines, calculer des moyennes de benchmark et ameliorer les visualisations Streamlit.